

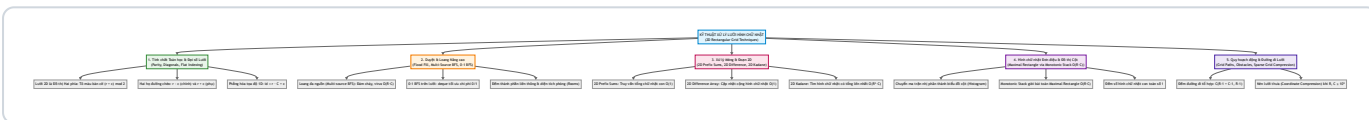
Chuyên đề: Các Kỹ thuật Xử lý Lưới Hình chữ nhật (2D Grid & Matrix Techniques)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy & Kiến trúc Chuyên đề



1. Tính chất Toán học & Biểu diễn Lưới 2D

Một lưới hình chữ nhật gồm R hàng và C cột là một trong những cấu trúc dữ liệu xuất hiện thường xuyên nhất trong lập trình thi đấu.

1.1. Phẳng hóa Tọa độ 2D $\leftrightarrow$ 1D (Flat Indexing)

Khi cần lưu trạng thái lưới vào mảng 1D hoặc cấu trúc Disjoint Set Union (DSU):

- Chuyển (r, c) thành chỉ số 1D:

$$\text{id} = r \times C + c \quad (0 \leq r < R, 0 \leq c < C)$$

- Khôi phục (r, c) từ chỉ số 1D:

$$r = \lfloor \text{id} / C \rfloor, \quad c = \text{id} \pmod{C}$$

1.2. Tính chẵn lẻ & Bàn cờ (Chessboard Parity)

Mỗi ô (r, c) có một độ chẵn lẻ xác định bởi:

$$\text{color}(r, c) = (r + c) \pmod{2}$$

🔴 Định lý Hai phía (Bipartite Property):

Mọi bước di chuyển trực giao (lên, xuống, trái, phải) luôn làm thay đổi màu của ô: từ $0 \rightarrow 1$ hoặc $1 \rightarrow 0$.

Do đó, **mọi lưới 2D đều là một đồ thị hai phía (Bipartite Graph)** không có chu trình độ dài lẻ!

Ứng dụng trong phòng thi:

- Để phủ kín bảng bằng các quân Domino 1×2 , số ô đen ($r + c \equiv 0$) bắt buộc phải bằng số ô trắng ($r + c \equiv 1$).
- Một quân Mã (Knight) di chuyển theo chữ L luôn nhảy giữa hai ô khác màu.

1.3. Hai Họ Đường chéo của Lưới

Trong các bài toán như **N-Queens**, chiếu sáng đèn, hay giám sát đường chéo:

1. **Đường chéo chính** (song song với đường chéo từ góc trên-trái xuống dưới-phải):

$$r - c = \text{const}$$

Giá trị $r - c$ nhận các giá trị từ $-(C - 1)$ đến $R - 1$.

Để đưa về chỉ số mảng không âm: $\text{diag}_1 = r - c + (C - 1) \in [0, R + C - 2]$.

2. **Đường chéo phụ** (song song với đường chéo từ góc trên-phải xuống dưới-trái):

$$r + c = \text{const}$$

Giá trị $r + c \in [0, R + C - 2]$.

2. Kỹ thuật Duyệt & Loang trên Lưới (Grid Traversal)

2.1. Loang Đa Nguồn (Multi-Source BFS)

Trong nhiều bài toán thực tế (đám cháy lây lan từ nhiều điểm, virus bùng phát, tìm khoảng cách tới chướng ngại vật gần nhất), ta cần tính khoảng cách ngắn nhất từ **mọi ô** tới **nguồn gần nhất**.

Thay vì chạy BFS từ từng nguồn một (tốn $O(K \cdot R \cdot C) \rightarrow \text{TLE}$):

1. Khởi tạo mảng khoảng cách `dist[r][c] = -1`.
2. Đẩy **tất cả** các điểm nguồn vào cùng một hàng đợi `std::queue<pair<int, int>>` với `dist[src_r][src_c] = 0`.
3. Chạy BFS một lần duy nhất. Độ phức tạp đạt mức tối ưu $O(R \cdot C)$.

```

vector<vector<int>> multiSourceBFS(int R, int C, const vector<pair<int, int>>& sources, const vector<vector<int>> dist(R, vector<int>(C, -1)));
queue<pair<int, int>> q;

for (auto [r, c] : sources) {
    dist[r][c] = 0;
    q.push({r, c});
}

const int dr[4] = {-1, 0, 1, 0};
const int dc[4] = {0, 1, 0, -1};

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    for (int i = 0; i < 4; ++i) {
        int nr = r + dr[i];
        int nc = c + dc[i];
        if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] != '#' && dist[nr][nc] == -1)
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
return dist;
}

```

2.2. 0-1 BFS trên Lưới (Di chuyển có Trọng số 0 hoặc 1)

Khi việc đi theo hướng cũ tốn chi phí 0, nhưng rẽ hướng tốn chi phí 1:

- Dùng `std::deque<pair<int, int>>`:
- Cạnh chi phí 0: Đẩy vào **đầu** hàng đợi (`push_front`).
- Cạnh chi phí 1: Đẩy vào **cuối** hàng đợi (`push_back`).
- Đảm bảo tính chất khoảng cách tăng dần của BFS mà không cần `std::priority_queue`, giảm độ phức tạp từ $O(R \cdot C \log(RC))$ xuống $O(R \cdot C)$.

3. Xử lý Mảng & Đoạn 2D (2D Prefix Sums & Difference)

3.1. Mảng Cộng dồn 2D (2D Prefix Sums)

Định nghĩa: $pref[r][c]$ là tổng các phần tử trong hình chữ nhật từ $(1, 1)$ đến (r, c) .

- **Xây dựng bảng trong** $O(R \cdot C)$:

$$pref[r][c] = pref[r-1][c] + pref[r][c-1] - pref[r-1][c-1] + a[r][c]$$

- Truy vấn tổng hình chữ nhật $[r_1..r_2] \times [c_1..c_2]$ trong $O(1)$:

$$\text{Sum} = \text{pref}[r_2][c_2] - \text{pref}[r_1 - 1][c_2] - \text{pref}[r_2][c_1 - 1] + \text{pref}[r_1 - 1][c_1 - 1]$$

3.2. Mảng Hiệu 2D (2D Difference Array)

Cần thực hiện Q thao tác: Cộng giá trị V vào tất cả các ô trong hình chữ nhật $[r_1..r_2] \times [c_1..c_2]$.

- Mỗi thao tác cập nhật trong $O(1)$ bằng 4 điểm mốc:

$$\begin{aligned} \text{diff}[r_1][c_1] &+= V \\ \text{diff}[r_1][c_2 + 1] &-= V \\ \text{diff}[r_2 + 1][c_1] &-= V \\ \text{diff}[r_2 + 1][c_2 + 1] &+= V \end{aligned}$$

- Sau khi hoàn thành Q thao tác, tính mảng cộng dồn 2D của diff để thu được bảng kết quả cuối cùng trong $O(R \cdot C)$.

3.3. 2D Kadane: Hình chữ nhật Con có Tổng Lớn nhất

Cho ma trận số nguyên kích thước $R \times C$ (có thể chứa số âm). Tìm hình chữ nhật con có tổng các phần tử lớn nhất.

- Thuật toán $O(R^2 \cdot C)$:

1. Cố định cặp hàng biên trên r_1 và biên dưới r_2 ($1 \leq r_1 \leq r_2 \leq R$).
2. Nén các hàng từ r_1 đến r_2 thành một mảng 1D kích thước C :

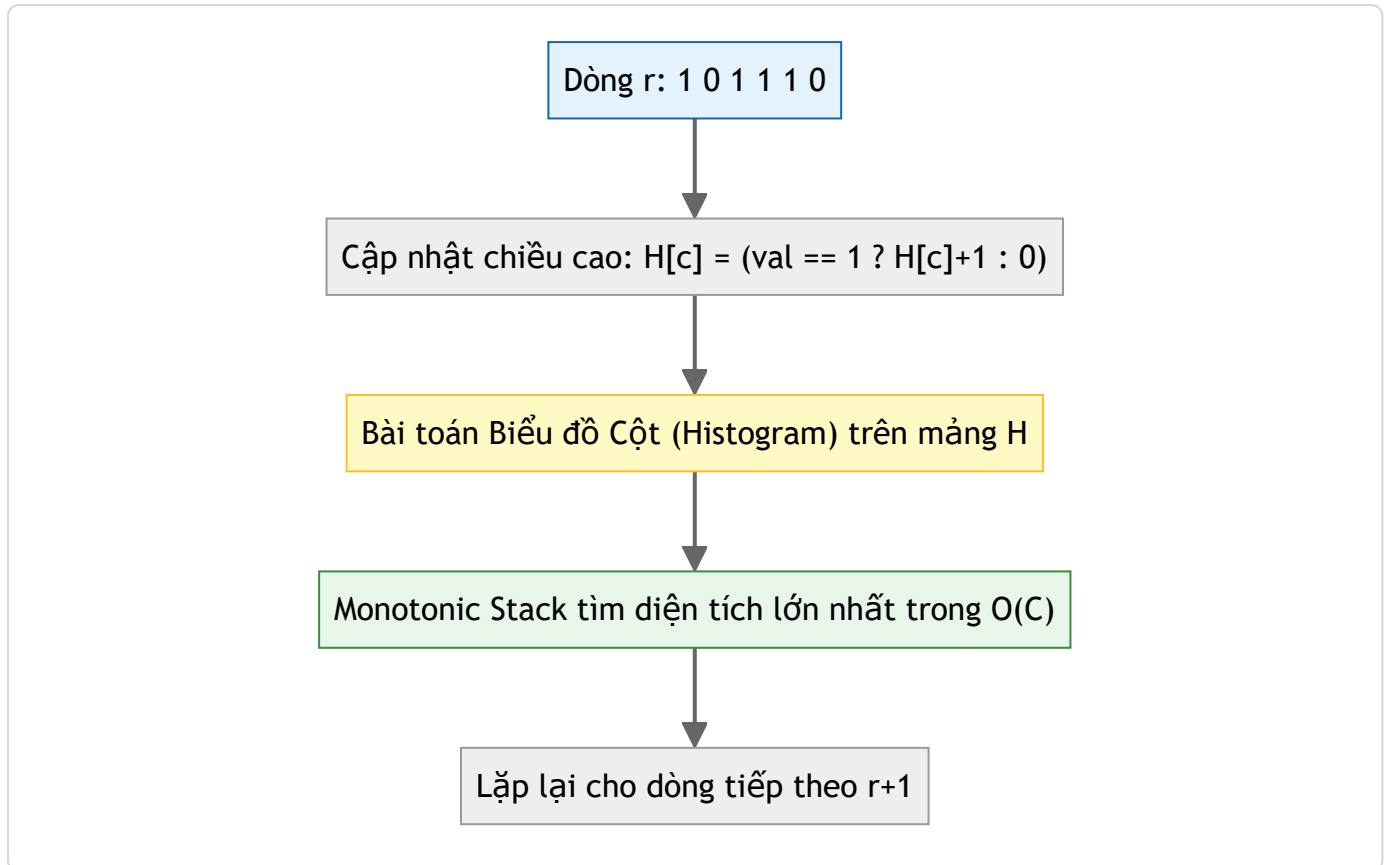
$$\text{colSum}[c] = \text{prefCol}[r_2][c] - \text{prefCol}[r_1 - 1][c]$$

3. Áp dụng thuật toán **Kadane 1D** trên mảng colSum để tìm đoạn con lớn nhất trong $O(C)$.

4. Tổng độ phức tạp: $\frac{R(R+1)}{2} \times O(C) = O(R^2 \cdot C)$.
-

4. Hình chữ nhật Đơn điệu: Maximal Rectangle of 1s trong $O(R \cdot C)$

Bài toán: Cho ma trận nhị phân kích thước $R \times C$ chỉ gồm các số 0 và 1. Tìm diện tích hình chữ nhật con lớn nhất chỉ toàn chứa số 1.



Thuật toán:

1. Duy trì mảng chiều cao $H[c]$ biểu thị số lượng số 1 liên tiếp hướng lên trên tính từ hàng hiện tại.
2. Với mỗi hàng r từ 0 đến $R - 1$:
3. Cập nhật: $H[c] = (\text{grid}[r][c] == '1' ? H[c] + 1 : 0)$.
4. Mảng H đóng vai trò là một **Biểu đồ cột (Histogram)**.
5. Sử dụng **Ngăn xếp đơn điệu (Monotonic Stack)** để tìm hình chữ nhật lớn nhất trong biểu đồ cột này trong thời gian $O(C)$.
6. Toàn bộ thuật toán chỉ mất $O(R \cdot C)$ **thời gian**, tối ưu tuyệt đối!

```

int maximalRectangle(const vector<string>& grid) {
    if (grid.empty()) return 0;
    int R = grid.size(), C = grid[0].size();
    vector<int> H(C, 0);
    int maxArea = 0;

    for (int r = 0; r < R; ++r) {
        for (int c = 0; c < C; ++c) {
            H[c] = (grid[r][c] == '1') ? H[c] + 1 : 0;
        }

        // Monotonic stack tính Largest Rectangle in Histogram
        vector<int> st;
        for (int c = 0; c <= C; ++c) {
            int curH = (c == C) ? 0 : H[c];
            while (!st.empty() && curH < H[st.back()]) {
                int h = H[st.back()];
                st.pop_back();
                int width = st.empty() ? c : (c - st.back() - 1);
                maxArea = max(maxArea, h * width);
            }
            st.push_back(c);
        }
    }
    return maxArea;
}

```

5. Kỹ thuật Nén Lưới Thưa (Sparse Grid Compression)

Nhiều bài toán đặt trên lưới khổng lồ $R, C \leq 10^9$, nhưng chỉ chứa $K \leq 1000$ điểm chướng ngại vật.

- Không thể khởi tạo mảng $R \times C$ trong bộ nhớ!

- **Giải pháp:**

1. Thu thập tất cả các tọa độ r_i và c_i xuất hiện trong đề bài.
2. Thêm các tọa độ lân cận: $r_i - 1, r_i, r_i + 1$ và $c_i - 1, c_i, c_i + 1$ để bảo toàn tính liên thông giữa các vùng trống.
3. Rời rạc hóa mảng tọa độ r và c (Coordinate Compression).
4. Lưới sau khi nén có kích thước tối đa $(4K) \times (4K) \approx 4000 \times 4000$, hoàn toàn vừa vặn trong bộ nhớ để chạy BFS / DSU thông thường!

6. Cạm bẫy Thường gặp & Bảng Kiểm tra Phòng thi

STT	Cạm bẫy phổ biến	Giải pháp khắc phục
1	Nhầm lẫn giữa Hàng R và Cột C khi duyệt	Quy ước bất biến: $r \in [0, R - 1]$ chạy theo chiều dọc, $c \in [0, C - 1]$ chạy theo chiều ngang. Mảng 2D luôn khai báo <code>grid[R][C]</code> .

STT	Cạm bẫy phổ biến	Giải pháp khắc phục
2	Tràn biên mảng khi kiểm tra lân cận	Luôn bọc điều kiện kiểm tra biên <code>isValid(nr, nc)</code> trước khi truy cập <code>grid[nr][nc]</code> .
3	Mảng hiệu 2D bị truy cập chỉ số $c_2 + 1$ vượt kích thước	Khởi tạo mảng <i>diff</i> có kích thước $(R + 2) \times (C + 2)$ với 1-based indexing.
4	Quên trường hợp toàn số âm trong 2D Kadane	Khởi tạo <code>maxSum = -INF</code> (hoặc giá trị phần tử lớn nhất) thay vì <code>maxSum = 0</code> .



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.